

Architecture Fonctionnelle

Conception de la solution fonctionnelle et de l'architecture informatique

Ce rapport présente l'architecture fonctionnelle de CareerAI, les acteurs du système, les parcours métier modélisés en BPMN, les choix architecturaux et les mesures de sécurité intégrées dès la phase de conception.

Projet	CareerAI — Plateforme SaaS de recherche d'emploi
Stack	Next.js 14 · Supabase · Groq IA · Resend
URL	https://www.careerai.live
GitHub	github.com/ARTHURNJOUONANG/Projet_Chat_board

1. Contexte et objectifs

CareerAI est une plateforme SaaS full-stack qui aide les candidats à structurer leur recherche d'emploi (CV, lettres de motivation, suivi) et à **automatiser une partie des candidatures** via des campagnes paramétrées. Un **espace recruteur** permet de gérer des postes, analyser des CV, envoyer des quiz techniques et classer les candidats.

Objectif métier	Description
Réduire les tâches répétitives	Recherche d'offres, envoi, relances automatisées
Contrôle utilisateur	Profil, plafonds d'envoi, consentement allow_auto_apply
Présélection recruteur	Scores, quiz techniques, classements par poste

2. Acteurs du système

Acteur	Rôle
Candidat	Crée un compte, configure son profil, lance des campagnes, consulte le chat IA et le suivi des candidatures.
Recruteur	Gère des postes, importe des candidats, génère et envoie des quiz, consulte les classements.
Systèmes externes	Supabase (auth/données), Groq (IA), Resend (emails), agrégateurs d'offres (Adzuna, LBA, France Travail).
Administrateur	Configure les variables d'environnement, applique les migrations SQL, planifie le cron des campagnes.

3. Besoins fonctionnels principaux

Côté candidat

Authentification (inscription, connexion, mot de passe oublié) · Assistant conversationnel (conseils carrière, génération de contenu) · Builder et export de CV, gestion PDF/DOCX · Campagnes (profil candidat, création jobs/kandi, exécution manuelle ou cron) · Suivi des candidatures et statistiques (analytics).

Côté recruteur

CRUD postes (job_postings) · Ajout de candidats (upload CV, analyse IA) · Génération et envoi de quiz par email (lien token unique) · Calcul de scores de pertinence et classements par poste.

Transversal

Internationalisation FR/EN (LanguageContext) · Application mobile Android via Capacitor (WebView).

4. Parcours BPMN

4.1 Lancer une campagne de candidatures

Le diagramme ci-dessous illustre le flux complet depuis la vérification de connexion jusqu'à l'enregistrement des candidatures, avec les deux types de campagne (jobs et kandi).

BPMN — Lancer une campagne de candidatures

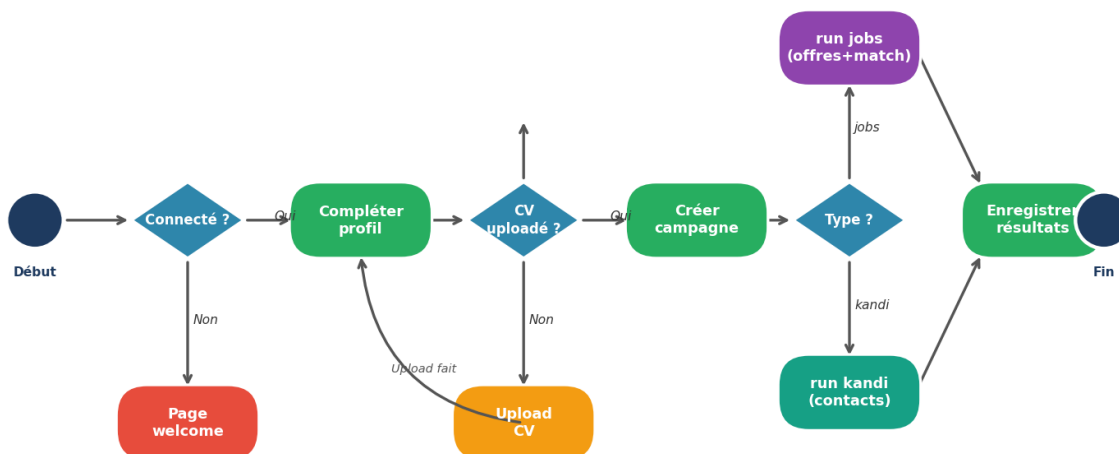


Figure 1 — BPMN : Lancer une campagne de candidatures

4.2 Envoyer un quiz à un candidat

Ce parcours illustre la séquence permettant au recruteur d'envoyer un lien de quiz personnalisé par email, avec gestion des cas d'erreur (quiz inactif, échec Resend).

BPMN — Envoyer un quiz à un candidat

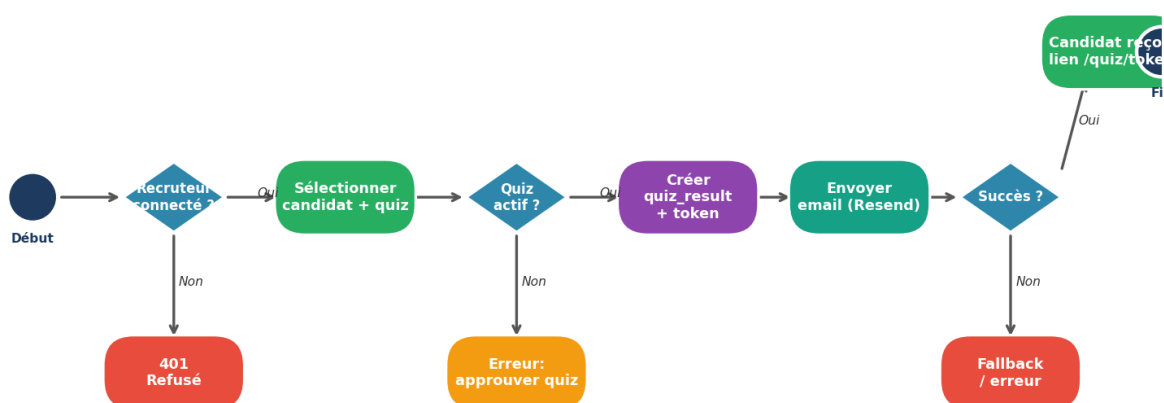


Figure 2 — BPMN : Envoyer un quiz à un candidat

5. Choix architecturaux et trade-offs

5.1 Monolithe Next.js vs Microservices

Option	Avantages	Inconvénients	Choix
Monolithe Next.js	Un déploiement, partage de types	Complexité de maintenance, scaling difficile	Revenant à la taille actuelle de l'équipe
Microservices	Isolation, scaling indépendant par service	Complexité ops, réseau, cohérence	Reporté si montée en charge significative

5.2 Supabase (BaaS) vs backend PostgreSQL custom

Option	Avantages	Inconvénients	Choix
Supabase (BaaS)	Auth, RLS, Storage intégrés — dépendance réduite	Limites de requêtes, limites de stockage	Retenu — V1 rapide
Backend custom	Contrôle total de l'infrastructure	Coût de développement auth/RLS	Retenu en V1

5.3 Campagnes : email + Puppeteer optionnel

Option	Avantages	Inconvénients	Choix
Email seul (Resend)	Simple, traçable, peu de maintenance	Pas de soumission sur tous les points	Par défaut
+ Puppeteer (opt.)	Soumission de formulaires web automatisés	Complexité (UI change), coût mémoire	Optionnel — désactivé si non configuré

6. Sécurité by Design

Les principes de sécurité sont intégrés dès la phase de conception, sans être considérés comme un ajout a posteriori.

Sécurité by Design — Mécanismes CareerAI



Principe : séparation client/serveur — les secrets ne quittent jamais Node.js

Figure 3 — Mécanismes de sécurité by Design

Mécanisme	Implémentation	Fichiers clés
-----------	----------------	---------------

Séparation client/serveur	Secrets jamais exposés au navigateur	.env.local — app/api/*
Authentification	Supabase Auth + cookies de session	middleware.js
Autorisation données	Row Level Security sur PostgreSQL	Policies user_id / recruter_id
Routes publiques limitées	Quiz par token, cron par secret	/api/candidate/quiz/{token}
Consentement campagnes	Flag allow_auto_apply avant envoi auto	candidate_profiles
Validation entrées	Contrôles côté API (email, fichiers, plafonds)	app/api/**/route.js